

Multi-Agent Reinforcement Learning for America’s Cup Regattas

Francesco Maria Fuligni
Mattia Furini
Mohamed Samir Haffoudhi
Daniele Silvestri

June 1, 2026

Abstract

This report presents a Multi-Agent Reinforcement Learning framework for a two-dimensional America’s Cup regatta simulator. The project extends a didactic sailing reinforcement-learning setting into a competitive PettingZoo ParallelEnv in which two autonomous boats learn continuous rudder and sail-trim control under stochastic wind, polar-performance constraints, and hybrid displacement/foiling dynamics. The agents are trained with Proximal Policy Optimization (PPO) through Stable-Baselines3 and SuperSuit vectorization. The available evidence consists of the trained model artifact, rendered trajectories, and exported training plots for mean episodic reward and mean Velocity Made Good (VMG). Since raw evaluation logs are not present in the repository, this report does not claim a measured finish rate; instead, it provides a code-grounded formulation of the environment and a qualitative analysis of the observed learning behavior.

1 Introduction

Autonomous sailing is a sequential decision-making problem in which control, navigation, and environmental adaptation are tightly coupled. A sailboat cannot move directly in arbitrary directions: its feasible velocity is constrained by the wind angle, the aerodynamic efficiency of the sails, hydrodynamic drag, and, in modern high-performance boats, the transition between displacement sailing and foiling. America’s Cup racing adds a further competitive layer: each boat must not only optimize its own progress around the course, but also react to an opponent while respecting tactical constraints and racing rules.

This project, *CoppaAmerica Multiagent*, implements a simplified but technically rich 2D simulator of an America’s Cup match race. The environment contains two agents, `red_boat` and `blue_boat`, a rectangular race field, an upwind/downwind course with mandatory gates, a stochastic wind field, continuous rudder and sail-trim actions, and a dense reward function designed to encourage progress, VMG, trim efficiency, foiling stability, and safe tactical interactions.

The work is conceptually connected to the course notebook *RL basics: learning how to sail*, where a single sailboat learns to reach a waypoint in a simplified Gymnasium environment using basic state variables, actions, rewards, polar behavior, and VMG. The present project generalizes that educational setting along four dimensions: it moves from a single-agent task to a competitive multi-agent environment, from simplified/discrete steering to continuous control, from a basic physical model to a hybrid displacement/foiling polar model, and from a fixed or simple wind model to stochastic spatio-temporal wind dynamics.

2 Background

2.1 Notation and Agent–Environment Interaction

Following the notation used in the course notes, random variables are denoted by uppercase letters, such as S_t , A_t , and R_t , while their realizations are denoted by lowercase letters, such as s_t , a_t , and r_t .

Expectations are taken with respect to the probability distribution induced jointly by the policy and the environment dynamics.

In Reinforcement Learning, an agent interacts with an environment over discrete time steps. At time t , the agent observes a state S_t , selects an action A_t , receives a reward R_t , and the environment transitions to a new state S_{t+1} . The interaction can be written as

$$S_t \xrightarrow{A_t} (S_{t+1}, R_t). \quad (1)$$

In stochastic environments, the next state and reward are not deterministic functions of the current state and action. They are sampled from environment-induced distributions:

$$S_{t+1} \sim P(\cdot \mid S_t, A_t), \quad R_t \sim R(\cdot \mid S_t, A_t). \quad (2)$$

2.2 Markov Decision Processes

The Markov property states that the future depends on the present state and not on the full history:

$$\Pr(X_{t+1} = j \mid X_t = i, X_{t-1}, \dots, X_0) = \Pr(X_{t+1} = j \mid X_t = i). \quad (3)$$

An MDP formalizes a sequential decision problem as a tuple

$$M = (\mathcal{S}, \mathcal{A}, P, r, \mu, \gamma), \quad (4)$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition probability, $r(s, a)$ is the reward function, μ is the initial state distribution, and $\gamma \in [0, 1]$ is the discount factor.

In the present simulator, each boat observes only a feature vector derived from the full simulator state. The implementation is therefore best interpreted as a multi-agent, partially observed control problem, but the learning interface follows the standard MDP formulation at the level of each agent's observation and action.

2.3 Policies, Trajectories, and Return

A policy defines how actions are selected from states. A deterministic policy has the form

$$A_t = \pi(S_t), \quad (5)$$

whereas a stochastic policy samples an action according to a conditional distribution:

$$A_t \sim \pi(\cdot \mid S_t). \quad (6)$$

For a finite horizon T , a trajectory can be written as

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T). \quad (7)$$

Under a stochastic policy π , the probability of such a trajectory is

$$p(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t). \quad (8)$$

The objective of reinforcement learning is not to maximize immediate reward alone, but cumulative future reward. The discounted return from time t is

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k}. \quad (9)$$

The goal is to find a policy that maximizes the expected discounted return:

$$J(\pi) = \mathbb{E}_{s_0 \sim \mu, \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, A_t) \right]. \quad (10)$$

2.4 Value Functions and Bellman Equation

The state-value function measures how much return is expected when starting from state s and following policy π :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s \right]. \quad (11)$$

The action-value function measures the expected return when first taking action a in state s and then following policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s, A_0 = a \right]. \quad (12)$$

The two functions are related by averaging action values under the policy:

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a). \quad (13)$$

The Bellman expectation equation expresses the recursive structure of value:

$$V^\pi(s) = \sum_a \pi(a \mid s) \left[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\pi(s') \right]. \quad (14)$$

Equivalently, the action-value function satisfies

$$Q^\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\pi(s'). \quad (15)$$

2.5 Model-Free Learning, Function Approximation, and PPO

In model-free reinforcement learning, the transition model P and reward model r are not assumed to be known analytically. Instead, the agent learns from sampled transitions (S_t, A_t, R_t, S_{t+1}) . With function approximation, value functions or policies are represented by parameterized models, typically neural networks. The temporal-difference error for a value approximation V_θ is

$$\delta_t = R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t), \quad (16)$$

and the corresponding squared loss is

$$L(\theta) = \frac{1}{2} (R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t))^2. \quad (17)$$

The America’s Cup simulator has continuous observations, continuous actions, and non-linear transition dynamics induced by wind, polar curves, foiling state, and multi-agent interaction. For this reason, the implementation uses Proximal Policy Optimization (PPO), as provided by Stable-Baselines3, as a practical model-free policy-gradient method suitable for continuous control. PPO is not used here to introduce additional theoretical machinery beyond the course notes; rather, it is the concrete algorithmic tool that maps the function-approximation perspective to a trainable continuous-control policy.

3 Methodology

3.1 Race Course and Multi-Agent Formulation

The environment is implemented as a `PettingZoo ParallelEnv` with two agents:

$$\mathcal{I} = \{\text{red_boat}, \text{blue_boat}\}. \quad (18)$$

The field has width $W = 1900$ m and length $L = 4100$ m. The top gate is located at $y = 3900$ m, the bottom gate at $y = 200$ m, and the gate width is 300 m. Each episode has a maximum duration of 1800 simulation steps with $\Delta t = 1$ s.

The race is decomposed into sequential legs. The boats start below the bottom gate, cross the start gate, sail upwind toward the top gate, perform a rounding maneuver, and then sail downwind toward the bottom gate. Gate violations, course violations, collisions, and excessive spinning can terminate the episode with severe penalties.

3.2 Observation Space

Each agent receives a normalized continuous observation vector

$$o_t^i \in [-1, 1]^{23}. \quad (19)$$

The vector contains normalized position, speed, heading, relative wind angle, local wind speed, distances and bearings to the active gate marks, current rudder and trim state, foiling state, active foil side, leg indicator, and opponent-relative features. With frame stacking configured as 4, the effective policy input is

$$\tilde{o}_t^i = (o_{t-3}^i, o_{t-2}^i, o_{t-1}^i, o_t^i) \in [-1, 1]^{92}. \quad (20)$$

Feature group	Observation components
Boat kinematics	x/W , y/L , speed divided by 50, and sin / cos encoding of heading.
Wind and controls	sin / cos encoding of the local relative wind angle, local wind speed divided by 30, rudder, sail trim mapped to $[-1, 1]$, foiling flag, and active foil side.
Course geometry	Normalized distances and relative bearings to the left and right marks, plus a signed leg indicator.
Opponent state	Relative opponent position, normalized opponent distance, and speed advantage.

Table 1: Observation structure used by each agent before frame stacking.

3.3 Action Space and Kinematics

The action space is two-dimensional and continuous:

$$a_t^i = (u_t^i, z_t^i) \in [-1, 1]^2, \quad (21)$$

where u_t^i is the rudder command and z_t^i is the sail-trim command. The trim action is mapped to a physical trim level by

$$\tau_t = \text{clip} \left(\frac{z_t + 1}{2}, 0, 1 \right). \quad (22)$$

The heading evolves according to a speed-dependent turn rate. Let v_t be the current speed in knots, $v_{\max} = 50$, $\rho_{\min} = 0.12$, and $\omega_{\max} = 25^\circ$. The effective turn factor is

$$\rho_t = \rho_{\min} + (1 - \rho_{\min}) \frac{v_t}{v_{\max}}, \quad (23)$$

and the heading update is

$$\psi_{t+1} = (\psi_t + u_t \rho_t \omega_{\max} \Delta t) \bmod 2\pi. \quad (24)$$

The boat position is updated using the simulated speed converted from knots to meters per second:

$$p_{t+1} = p_t + 0.514 v_{t+1} \Delta t \begin{bmatrix} \cos \psi_{t+1} \\ \sin \psi_{t+1} \end{bmatrix}. \quad (25)$$

3.4 Stochastic Wind Field

The wind model contains a global base wind and a spatial perturbation grid. At reset, the base wind direction is initialized near north-to-south flow, with random angular offset, and the base wind speed is sampled from $[15, 22]$ knots. At each step,

$$\Theta_{t+1} = (\Theta_t + \varepsilon_t^\Theta) \bmod 2\pi, \quad \varepsilon_t^\Theta \sim \mathcal{U}(-0.02, 0.02), \quad (26)$$

$$U_{t+1} = \text{clip}(U_t + \varepsilon_t^U, 15, 22), \quad \varepsilon_t^U \sim \mathcal{U}(-0.3, 0.3). \quad (27)$$

Local gusts are represented by a 10×10 grid. For each cell (m, n) , direction and speed perturbations evolve as

$$D_{t+1}^{m,n} = \text{clip} \left(0.95 D_t^{m,n} + \eta_t^{D,m,n}, -2.5(0.15), 2.5(0.15) \right), \quad (28)$$

$$S_{t+1}^{m,n} = \text{clip} \left(0.95 S_t^{m,n} + \eta_t^{S,m,n}, -2.5(2.0), 2.5(2.0) \right), \quad (29)$$

where $\eta_t^{D,m,n} \sim \mathcal{N}(0, 0.015^2)$ and $\eta_t^{S,m,n} \sim \mathcal{N}(0, 0.2^2)$. The local wind experienced by the boat is obtained by bilinear interpolation of the four nearest grid cells and then added to the base wind.

3.5 Polar-Speed Model and Foiling Dynamics

The code computes a relative wind angle $\phi_t \in [0, 180]$ degrees from the local wind direction and boat heading. The theoretical polar speed is

$$v_t^{\text{base}} = \min(q_{\kappa_t}(\phi_t) U_t^{\text{local}}, 50), \quad (30)$$

where $\kappa_t \in \{D, F\}$ denotes displacement or foiling mode. In foiling mode, the multiplier is

$$q_F(\phi) = \begin{cases} 0, & 0 \leq \phi < 45, \\ 1.0 + 0.15(\phi - 45), & 45 \leq \phi < 55, \\ 2.5 + 0.024(\phi - 55), & 55 \leq \phi < 100, \\ 3.6 + 0.015(\phi - 100), & 100 \leq \phi < 140, \\ 4.2 - 0.03(\phi - 140), & 140 \leq \phi < 170, \\ 3.3 - 0.25(\phi - 170), & 170 \leq \phi \leq 180. \end{cases} \quad (31)$$

In displacement mode, the multiplier is

$$q_D(\phi) = \begin{cases} 0, & 0 \leq \phi < 35, \\ 0.3 + 0.03(\phi - 35), & 35 \leq \phi < 50, \\ 0.75 + 0.015(\phi - 50), & 50 \leq \phi < 110, \\ 1.65 - 0.01(\phi - 110), & 110 \leq \phi < 140, \\ 1.35 - 0.005(\phi - 140), & 140 \leq \phi \leq 180. \end{cases} \quad (32)$$

The optimal trim $\tau^*(\phi, \kappa)$ is computed by linear interpolation over mode-specific trim points. The trim efficiency is Gaussian:

$$e_\tau = \exp \left[-\frac{1}{2} \left(\frac{|\tau - \tau^*|}{\sigma_\kappa} \right)^2 \right], \quad \sigma_F = 0.12, \quad \sigma_D = 0.16. \quad (33)$$

The speed multiplier induced by trim is

$$m_\tau(e_\tau, \kappa) = \begin{cases} 0.60 + 0.65e_\tau, & \kappa = F, \\ 0.72 + 0.42e_\tau, & \kappa = D. \end{cases} \quad (34)$$

The target speed is

$$v_t^{\text{target}} = \min(v_t^{\text{base}} m_\tau(e_\tau, \kappa_t), 50). \quad (35)$$

Actual speed follows an inertia update:

$$v_{t+1} = I_{\kappa_t} v_t + (1 - I_{\kappa_t}) v_t^{\text{target}}, \quad I_F = 0.98, \quad I_D = 0.85. \quad (36)$$

The boat enters foiling mode when $v_{t+1} \geq 18$ knots and drops out of foiling when $v_{t+1} < 15$ knots.

3.6 Reward Shaping

The reward is dense and multi-objective. Let g_t be the active target, p_t the boat position, and

$$d_t = \|p_t - g_t\|_2, \quad \Delta d_t = d_{t-1} - d_t. \quad (37)$$

The progress reward is asymmetric: losing distance is penalized more strongly than gaining distance is rewarded. With $c_t = 1.8$ in foiling mode and $c_t = 0.65$ otherwise,

$$r_t^{\text{prog}} = \begin{cases} c_t \Delta d_t, & \Delta d_t \geq 0, \\ 1.8 c_t \Delta d_t, & \Delta d_t < 0. \end{cases} \quad (38)$$

VMG is the projection of boat velocity onto the direction of the active target:

$$\text{VMG}_t = v_t \begin{bmatrix} \cos \psi_t \\ \sin \psi_t \end{bmatrix}^\top \frac{g_t - p_t}{\|g_t - p_t\|_2}, \quad \widehat{\text{VMG}}_t = \text{clip} \left(\frac{\text{VMG}_t}{50}, -1, 1 \right). \quad (39)$$

With $\lambda_t = 1.6$ on the upwind leg and $\lambda_t = 1.25$ otherwise, the VMG term is

$$r_t^{\text{vmg}} = 8.5 \lambda_t \max(\widehat{\text{VMG}}_t, 0) + 4.0 \lambda_t \min(\widehat{\text{VMG}}_t, 0). \quad (40)$$

Let β_t be the bearing from the boat to the target and

$$h_t = \frac{|\text{wrap}(\beta_t - \psi_t)|}{\pi}. \quad (41)$$

The heading-alignment term is

$$r_t^{\text{head}} = 1.2(1 - h_t) - 3.0 \max(h_t - 0.75, 0). \quad (42)$$

The rudder penalty discourages aggressive and oscillatory steering:

$$r_t^{\text{rudder}} = -2|u_t|^3 - 5(u_t - u_{t-1})^2. \quad (43)$$

Trim and foiling rewards are

$$r_t^{\text{trim}} = 8.5(e_{\tau,t} - 0.72) - 5.5|\tau_t - \tau_{t-1}|^{1.5} - 18 \max(|\tau_t - \tau_t^*| - \xi_t, 0) \mathbb{I}[v_t > 10], \quad (44)$$

$$r_t^{\text{vmg-trim}} = 5 \max(\widehat{\text{VMG}}_t, 0)(e_{\tau,t} - 0.50), \quad (45)$$

where $\xi_t = 0.18$ on the upwind leg and $\xi_t = 0.24$ otherwise. If a boat drops from foiling to displacement mode, it receives

$$r_t^{\text{drop}} = -40. \quad (46)$$

The remaining speed and foiling terms are

$$r_t^{\text{foil}} = \begin{cases} 1.1(v_t - 15), & \kappa_t = F, \\ -1.5, & \kappa_t = D, \end{cases} \quad r_t^{\text{low}} = -3 \mathbb{I}[v_t < 15], \quad (47)$$

and

$$r_t^{\text{speed}} = -0.35 \max(v^{\text{leg}} - v_t, 0) + 0.28 v_t \mathbb{I}[\Delta d_t > 0], \quad (48)$$

where $v^{\text{leg}} = 20$ knots on the upwind leg and 24 knots otherwise.

The multi-agent tactical advantage compares the two boats' distances to their active targets:

$$r_t^{\text{tac}} = 1.5 \text{clip} \left(\frac{d_t^{\text{opp}} - d_t}{\sqrt{W^2 + L^2}}, -1, 1 \right). \quad (49)$$

The course-direction shaping term rewards movement along the appropriate vertical direction:

$$r_t^y = \begin{cases} 0.15(y_t - y_{t-1}), & \ell_t \in \{0, 1\}, \\ -0.15(y_t - y_{t-1}), & \ell_t \notin \{0, 1\}. \end{cases} \quad (50)$$

Collision avoidance is implemented through near-collision, time-to-collision, and hard-collision penalties. If d_t^{ij} is the distance between two boats and $d_t^{ij} < 40$, the near-collision penalty is

$$p_t^{\text{near}} = 3 \frac{40 - d_t^{ij}}{40}. \quad (51)$$

If the closing speed c_t^{ij} is positive, the time-to-collision is

$$\text{TTC}_t = \frac{d_t^{ij}}{c_t^{ij}}. \quad (52)$$

For $\text{TTC}_t < 4$, the predictive penalty is

$$p_t^{\text{ttc}} = 5 \left(1 - \frac{\text{TTC}_t}{4} \right) \frac{40 - d_t^{ij}}{40}. \quad (53)$$

If $d_t^{ij} < 20$, the overlap penalty is

$$p_t^{\text{coll}} = 20 \frac{20 - d_t^{ij}}{20}, \quad (54)$$

with a multiplier 1.6 applied to the port-tack boat on opposite tacks. Hard violations such as severe collision, missed gates, out-of-bounds movement, wrong-course movement, failed rounding, and spin violation receive a terminal penalty of -10000 .

Gate bonuses are sparse but important:

$$b_t = 400 \mathbb{I}[\text{valid start gate}] + 500 \mathbb{I}[\text{valid top rounding}] + B_t^{\text{finish}}, \quad (55)$$

where

$$B_t^{\text{finish}} = \begin{cases} 2500 + 1000 \frac{1800 - t}{1800}, & \text{first finisher,} \\ 700, & \text{second finisher,} \\ 0, & \text{otherwise.} \end{cases} \quad (56)$$

Combining the dense components, gate events, and terminal events, the step reward is

$$\begin{aligned} r_t = & r_t^{\text{prog}} + r_t^{\text{vmg}} + r_t^{\text{head}} + r_t^{\text{rudder}} + r_t^{\text{trim}} + r_t^{\text{vmg-trim}} + r_t^{\text{foil}} + r_t^{\text{low}} + r_t^{\text{speed}} \\ & + r_t^{\text{tac}} + r_t^y - 0.25 - p_t^{\text{near}} - p_t^{\text{ttc}} - p_t^{\text{coll}} + b_t + p_t^{\text{terminal}}. \end{aligned} \quad (57)$$

4 Experimental Setup and Results

4.1 Training Infrastructure

The training pipeline uses Stable-Baselines3 PPO. The PettingZoo environment is wrapped with SuperSuit, converted to a vectorized environment, monitored with **VecMonitor**, and augmented with **VecFrameStack**. The configuration is read from `config.yaml`; therefore the hyperparameters in Table 2 reflect the actual project configuration rather than earlier draft values.

4.2 Available Evidence

The repository contains a trained model artifact, exported plots, and a rendered demonstration video, but it does not contain raw evaluation logs, CSV summaries, or TensorBoard event files. Consequently, the report does not present measured finish-rate tables or DNF percentages. This is a deliberate methodological choice: reporting quantitative outcome distributions without raw evaluation data would be less rigorous than explicitly limiting the conclusions to the available artifacts.

The available curves indicate that the optimization process produced a policy with improved reward and VMG behavior over training. The rendered trajectory further shows that the learned policy can generate non-trivial navigation behavior around the course, including gate-oriented sailing and interaction with the opponent.

Parameter	Configured value
Algorithm	PPO
Total configured training steps	2,000,000
Parallel environments (n_envs)	16
Agents per environment	2
Frame stacking	4 frames
Effective input dimension	$23 \times 4 = 92$
Network architecture	MLP [256, 256] for policy and value networks
Learning rate	2×10^{-4}
Rollout steps (n_steps)	4096
Batch size	1024
Optimization epochs	10
Discount factor γ	0.99
GAE parameter λ	0.95
PPO clip range	0.2
Entropy coefficient	0.01
Success-window size	100 episodes
Success threshold	0.95

Table 2: Training configuration extracted from `config.yaml`.

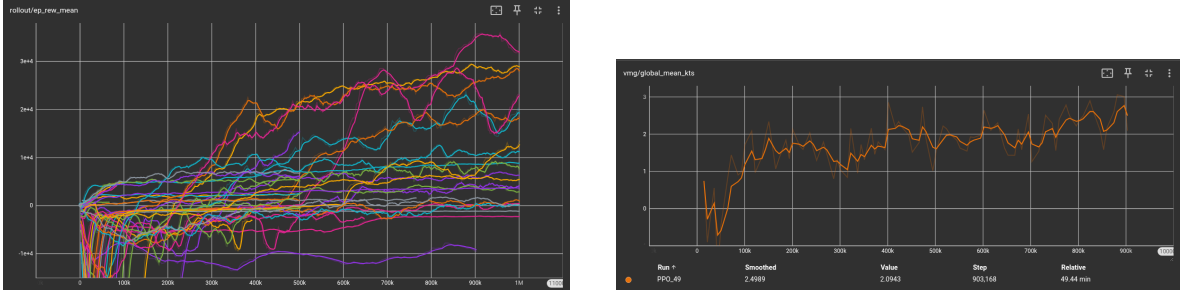


Figure 1: Available training plots. The left plot reports mean episodic reward; the right plot reports global mean VMG in knots. These figures support a qualitative discussion of learning dynamics, but not a standalone finish-rate estimate.

Outcome category	Meaning in the simulator
finished_first, finished_second	Boat crosses the bottom finish gate after completing the course.
missed_start_gate, missed_top_gate, missed_bottom_gate	Boat crosses a required gate line outside the legal gate interval.
failed_rounding	Boat fails to complete the mandatory rounding segment after re-tries.
wrong_course, out_of_bounds	Boat violates course direction or exits the valid field.
collision collision_port_ tack_violation	Severe collision, with Rule 10 asymmetry on opposite tacks.
spin_violation, timeout	Degenerate turning behavior or maximum episode length reached.

Table 3: Environment-defined outcome categories available for future quantitative evaluation.

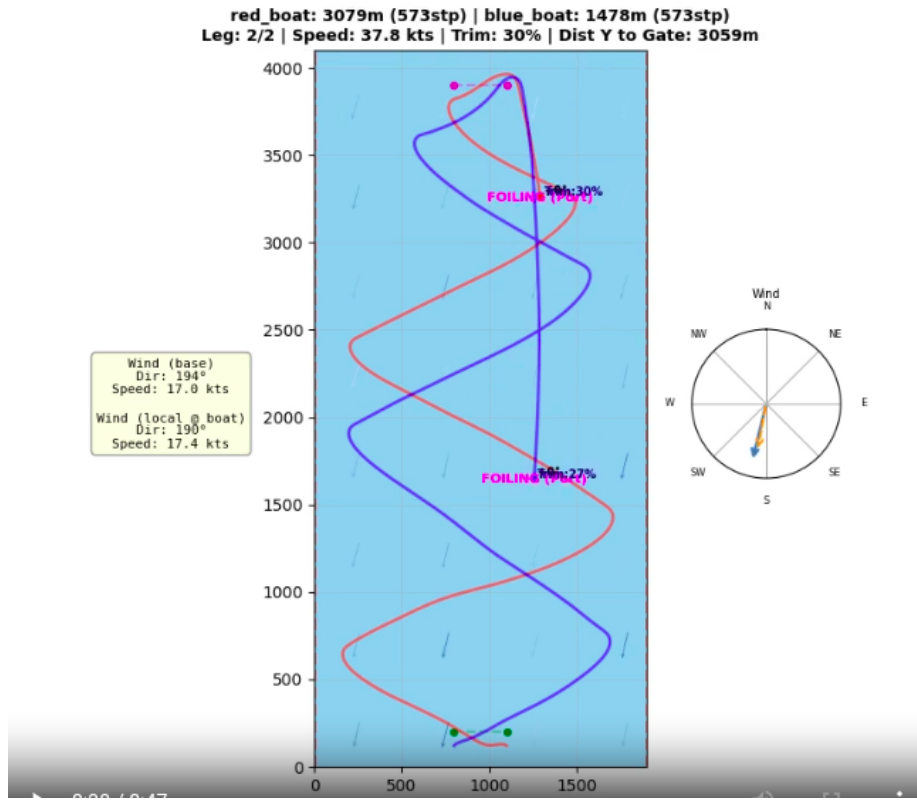


Figure 2: Rendered simulation frame from the available demonstration. The visualization is used as qualitative evidence of learned navigation behavior, not as a statistical evaluation over multiple seeds.

4.3 Implemented Outcome Categories

Although empirical counts are not available, the environment implements a detailed set of terminal outcomes. These categories can be interpreted as DNF-style failure modes for future evaluation:

4.4 Comparison with the Course Notebook

The class notebook introduces a didactic sailing task with a single agent, a simplified state representation, basic actions, reward based on target distance and speed, and a simplified polar/VMG model. The present project preserves the same conceptual core—learning a sailing policy from interaction—but substantially increases the complexity of the environment. It introduces two simultaneous agents, opponent-relative observations, PettingZoo parallel execution, continuous rudder and sail-trim actions, stochastic wind, gate logic, collision handling, Rule 10-inspired penalties, and a hybrid displacement/foiling performance model.

This progression is important pedagogically: the project remains grounded in the course’s MDP and model-free RL formulation, but demonstrates how the same concepts scale to a richer multi-agent control problem.

5 Conclusions

This report formalized the *CoppaAmerica Multiagent* simulator as a model-free reinforcement-learning problem with continuous observations, continuous actions, stochastic wind, hybrid boat dynamics, and multi-agent tactical interaction. The theoretical background follows the course progression from agent-environment interaction to MDPs, policies, return, value functions, Bellman equations, model-free learning, and function approximation. PPO is then used as the practical continuous-control algorithm.

The main strength of the project is the integration of several interacting mechanisms: polar performance, trim efficiency, foiling transitions, VMG optimization, tactical advantage, gate sequencing, and collision/rule penalties. The main limitation of the current report is empirical: the repository does not contain raw evaluation logs, so finish rates and DNF distributions cannot be stated rigorously. Future work should add a deterministic evaluation script that runs a fixed set of seeds, records terminal reasons, computes finish rates and DNF categories, and exports a reproducible CSV table for inclusion in the report.

References

- [1] Francesco Pivi. *Reinforcement Learning Introduction*. Course notes, 2025.
- [2] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347, 2017.
- [3] J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, and others. *Petting-Zoo: Gym for Multi-Agent Reinforcement Learning*. Advances in Neural Information Processing Systems, 2021.
- [4] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, Noah Dornmann, and others. *Stable-Baselines3: Reliable Reinforcement Learning Implementations*. Journal of Machine Learning Research, 22(268):1–8, 2021.